



南开大学
Nankai University

《软件测试与维护》大作业报告

(2025~2026 学年第二学期)

实验名称：基于 SockShop 的微服务部署测试与异常检测

学 院：软件学院

小组成员：田皓天、刘涵博、姜弘毅、欧阳东俊、李宗杭、柯瀚

指导老师：张圣林

2026 年 6 月 19 日

目录

1 微服务系统部署与环境搭建	1
1 实验背景	1
2 部署步骤	1
2.1 Docker 环境准备	1
2.2 Minikube 集群搭建	1
2.3 SockShop 部署	2
2 监控系统搭建与数据采集	4
1 Prometheus + Grafana 监控部署	4
1.1 Prometheus 部署	4
1.2 Grafana 仪表盘配置	4
2 监控数据采集方案	5
2.1 采集指标类别	5
2.2 采集策略	6
3 功能与性能测试	7
1 Selenium 功能自动化测试	7
1.1 测试环境	7
1.2 测试用例设计	7
1.3 测试结果	7
2 JMeter/API 性能测试	8
2.1 测试设计	8
2.2 测试结果与分析	9
4 故障注入实验	12
1 ChaosMesh 概述	12
2 故障注入方案	12
2.1 Pod Kill 故障	12
2.2 CPU 压力故障	13
2.3 内存压力故障	13

3 故障注入实验结果	13
3.1 Pod Kill 实验结果	13
3.2 CPU 压力实验结果	13
3.3 内存压力实验结果	13
5 异常检测论文算法复现	15
1 LogAnomaly 算法复现	15
1.1 算法原理	15
1.2 实现方案	15
1.3 实验结果	16
2 SR-CNN 算法复现	17
2.1 算法原理	17
2.2 实现方案	17
2.3 实验结果	18
2.4 补充：在真实 KPI 数据上的测试与分析	19
3 算法对比分析	19
6 总结与心得体会	21
1 大作业总结	21
2 实验中的挑战与解决方案	21
2.1 SockShop 购物车与订单服务无法正常使用	21
2.2 监控数据采集	22
2.3 算法复现中的数据挑战	22
3 心得体会	22
A 附录 项目仓库地址	23
B 附录 团队成员分工表	24

一 微服务系统部署与环境搭建

1 实验背景

随着云计算和容器技术的快速发展，微服务架构已成为构建大规模分布式系统的主流范式。SockShop 是一个经典的微服务演示应用，由 Weaveworks 公司开发，模拟了一个在线袜子商店的完整业务流程。该系统由多个独立的微服务组成，包括前端服务 (front-end)、商品目录服务 (catalogue)、购物车服务 (carts)、订单服务 (orders)、支付服务 (payment)、用户服务 (user)、物流服务 (shipping)、消息队列 (queue-master) 以及 RabbitMQ 消息中间件等。每个微服务都有独立的数据库，通过 REST API 和消息队列进行通信。

本次大作业选择 SockShop 作为目标系统，在 Minikube 本地 Kubernetes 集群上完整实践了从部署、监控、测试到故障注入和 AI 异常检测的全流程。

2 部署步骤

2.1 Docker 环境准备

Docker 是本项目的基础运行时环境。Windows 系统下通过 WSL2 子系统安装 Docker Desktop，安装完成后通过以下命令验证：

```
1 docker --version
2 docker run hello-world
```

2.2 Minikube 集群搭建

Minikube 是 Kubernetes 的轻量化版本，用于在本地模拟 K8s 集群环境。安装 Minikube 后，使用以下命令启动集群：

```
1 minikube start --driver=docker --cpus=4 --memory=6g --force
```

集群启动后使用 kubectl 工具进行管理，通过以下命令验证集群状态：

```
1 minikube status
2 kubectl get nodes
```

2.3 SockShop 部署

从 GitHub 克隆 SockShop 项目并部署到 Minikube 集群：

```
1 git clone https://github.com/microservices-demo/microservices-demo
2 cd microservices-demo
3 kubectl create -f deploy/kubernetes/manifests/00-sock-shop-ns.yaml -f
  deploy/kubernetes/manifests
```

部署完成后，SockShop 系统的 14 个微服务 Pod 全部正常运行，其架构如表1.1所示。

服务名称	类型	端口	功能描述
front-end	NodePort	80:30001	前端 Web 界面
catalogue	ClusterIP	80	商品目录服务
catalogue-db	ClusterIP	3306	商品数据库 (MySQL)
carts	ClusterIP	80	购物车服务
carts-db	ClusterIP	27017	购物车数据库 (MongoDB)
orders	ClusterIP	80	订单服务
orders-db	ClusterIP	27017	订单数据库 (MongoDB)
payment	ClusterIP	80	支付服务
user	ClusterIP	80	用户服务
user-db	ClusterIP	27017	用户数据库 (MongoDB)
shipping	ClusterIP	80	物流服务
queue-master	ClusterIP	80	消息队列管理
rabbitmq	ClusterIP	5672/9090	RabbitMQ 消息中间件
session-db	ClusterIP	6379	会话数据库 (Redis)

表 1.1: SockShop 微服务架构

```
PS D:\Kubernetes\projects\microservices-demo> kubectl get pods -n sock-shop
NAME                                READY   STATUS    RESTARTS   AGE
carts-b6c5c87f9-j464w              1/1     Running   0           23m
carts-db-885756899-vfswz           1/1     Running   0           9m19s
catalogue-744d46f784-2np6h         1/1     Running   0           23m
catalogue-db-6d4c747cc6-qvf7j      1/1     Running   0           23m
front-end-87ff7b849-fppxj          1/1     Running   3 (3m37s ago)  23m
orders-6cfcf6794c-4fhrt            1/1     Running   0           23m
orders-db-5ccf6699df-4kv5l         1/1     Running   0           18s
payment-7588d7df-xrjqg             1/1     Running   0           23m
queue-master-f75b7664f-jpmpmt      1/1     Running   0           23m
rabbitmq-fcb976dd-fgl4j            2/2     Running   0           23m
session-db-5cc78648ff-k9zng        1/1     Running   0           23m
shipping-6ddcd77687-kcxvx          1/1     Running   0           23m
user-9c64594b7-ggs26              1/1     Running   0           23m
user-db-5b8fd7455-4xms2            1/1     Running   0           23m
```

图 1.1: SockShop Pod 运行状态

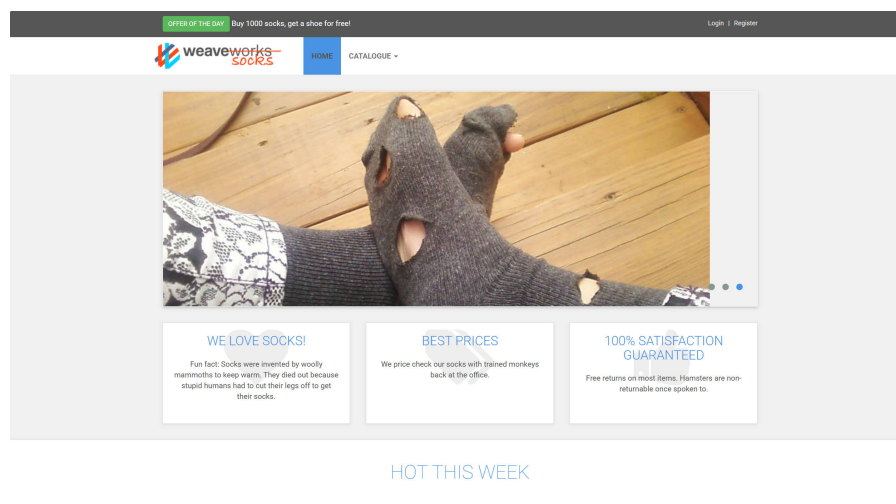


图 1.2: SockShop 前端界面

二 监控系统搭建与数据采集

1 Prometheus + Grafana 监控部署

在 SockShop 成功部署的基础上，使用 microservices-demo 项目中自带的监控配置文件搭建 Prometheus+Grafana 监控体系。

1.1 Prometheus 部署

执行以下命令部署监控组件：

```
1 kubectl create -f ./deploy/kubernetes/manifests-monitoring
```

部署后通过 Prometheus 的 NodePort 服务访问其 Web 界面。Prometheus 通过 kube-state-metrics 和 node-exporter 两个组件分别采集 Kubernetes 集群状态指标和节点级硬件资源指标。

1.2 Grafana 仪表盘配置

Grafana 通过 NodePort 服务提供仪表盘可视化展示。使用默认账号 admin/admin 登录后，添加 Prometheus 为数据源，导入仪表盘模板（ID 12633 用于 Linux 主机详情）。

Grafana 仪表盘可展示以下关键监控指标：

- CPU 使用率
- 内存使用情况
- 磁盘读写速率
- 网络收发流量



图 2.1: Prometheus 监控界面



图 2.2: Grafana 仪表盘

2 监控数据采集方案

为了支持后续的论文算法复现，我们设计了系统化的监控数据采集方案：

2.1 采集指标类别

1. 容器级指标：CPU 使用率、内存使用量、网络收发包速率、磁盘读写速率
2. Pod 级指标：Pod 就绪状态、容器重启次数
3. 节点级指标：节点 CPU 使用率、节点内存使用率、节点磁盘 I/O、节点网络带宽

4. **应用级指标**: HTTP 请求延迟、HTTP 状态码、API 响应时间
5. **日志数据**: 各 Pod 的容器日志

2.2 采集策略

通过 Python 脚本调用 Prometheus API 和 kubectl 命令，分别在以下两种状态下进行数据采集：

- **正常状态 (Baseline)**: 系统正常运行时期，采集 3 轮快照，每轮间隔 60 秒
- **故障状态 (Fault)**: 注入 Pod Kill、CPU Stress、Memory Stress 三种故障后，每轮采集 3 轮快照

采集数据以 JSON 格式保存原始快照，同时导出为 CSV 格式供算法直接读取使用。

三 功能与性能测试

1 Selenium 功能自动化测试

1.1 测试环境

- Python 3.12.11 + Selenium 3.17.0
- Microsoft Edge 浏览器
- 测试目标：SockShop 前端

1.2 测试用例设计

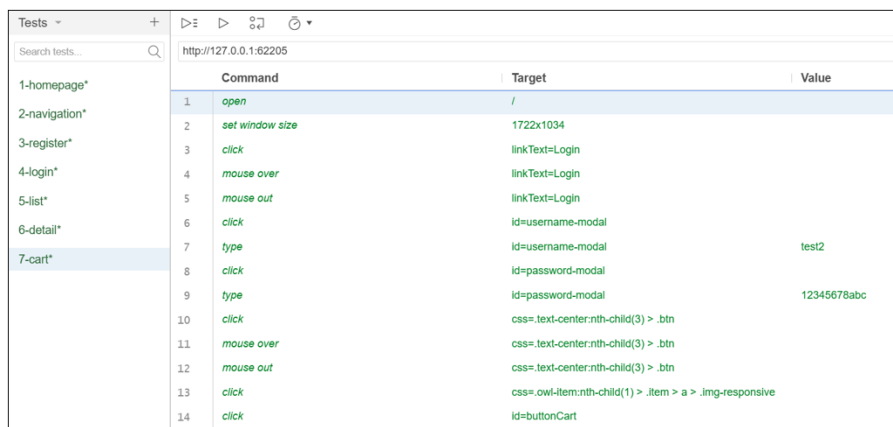
编写了覆盖 SockShop 核心功能流程的自动化测试脚本，具体测试用例如表3.1所示。

编号	测试用例	测试内容	预期结果
TC-01	首页加载	访问首页，验证页面标题和内容	页面标题为"WeaveSocks"
TC-02	页面导航	验证首页导航功能正常	导航成功
TC-03	用户注册	填写注册表单并提交	注册成功
TC-04	用户登录	使用注册账号登录	登录成功
TC-05	商品目录浏览	首页商品动态加载	商品内容正确显示
TC-06	商品详情查看	访问 detail.html 页面	商品详细信息显示
TC-07	购物车操作	添加商品到购物车	购物车数据正确更新

表 3.1: Selenium 功能测试用例

1.3 测试结果

所有 7 项测试全部通过，通过率 100%。



Command	Target	Value
1 open	/	
2 set window size	1722x1034	
3 click	linkText=Login	
4 mouse over	linkText=Login	
5 mouse out	linkText=Login	
6 click	id=username-modal	
7 type	id=username-modal	test2
8 click	id=password-modal	
9 type	id=password-modal	12345678abc
10 click	css=.text-center:nth-child(3) > .btn	
11 mouse over	css=.text-center:nth-child(3) > .btn	
12 mouse out	css=.text-center:nth-child(3) > .btn	
13 click	css=.owl-item:nth-child(1) > .item > a > .img-responsive	
14 click	id=buttonCart	

图 3.1: Selenium 自动化测试执行结果

2 JMeter/API 性能测试

2.1 测试设计

使用 JMeter 进行 API 性能测试，模拟不同负载级别下的用户并发访问。测试的 API 端点包括：

- GET / （首页）
- GET /catalogue （商品目录）
- GET /catalogue/\${productId} （商品详情）
- GET /register （注册）
- GET /login （登录）
- GET /cart （购物车）

设置了三种负载级别：

- 低负载：10 并发用户，60 次请求
- 中负载：50 并发用户，300 次请求
- 高负载：100 并发用户，600 次请求

2.2 测试结果与分析

系统在所有负载级别和测试接口下均保持 0% 错误率，展现出极佳的基础稳定性。测试采用 APDEX 指标量化用户体验，设定容忍阈值 $T=500\text{ms}$ ，沮丧阈值 $F=1500\text{ms}$ ，测试结果汇总如下表所示。

测试名称	平均 (ms)	P90(ms)	P99(ms)	吞吐量 (req/s)	APDEX
低负载 (10 并发用户)					
首页访问-低负载	10.90	33.10	35.00	1.12	1.000
浏览商品详情-低负载	51.60	67.20	68.00	1.22	1.000
浏览目录页-低负载	52.90	67.60	68.00	1.22	1.000
用户注册-低负载	64.90	178.60	188.00	1.12	1.000
加入购物车-低负载	139.70	376.70	411.00	1.22	0.950
用户登录-低负载	139.70	666.30	725.00	1.12	0.950
整体-低负载	64.95	91.60	725.00	4.23	0.992
中负载 (50 并发用户)					
首页访问-中负载	15.26	21.70	123.00	5.12	1.000
浏览商品详情-中负载	66.08	70.90	409.00	2.83	1.000
浏览目录页-中负载	73.10	163.60	407.00	2.83	1.000
用户注册-中负载	963.44	3385.90	4502.00	3.78	0.680
加入购物车-中负载	1491.00	4097.90	5499.00	2.83	0.570
用户登录-中负载	2939.48	7214.50	9289.00	2.84	0.330
整体-中负载	924.73	3969.00	8272.27	13.10	0.763
高负载 (100 并发用户)					
首页访问-高负载	8.22	13.00	63.66	10.14	1.000
浏览商品详情-高负载	34.22	62.90	70.00	4.07	1.000
浏览目录页-高负载	34.74	58.90	69.96	4.07	1.000
用户注册-高负载	2085.56	4761.20	10029.11	5.68	0.360
加入购物车-高负载	2955.62	6520.70	12366.73	4.01	0.295
用户登录-高负载	5508.38	9564.40	15043.16	3.93	0.035
整体-高负载	1771.12	5764.80	10578.06	18.87	0.615

表 3.2: API 性能测试结果汇总

性能分析：

- 读写接口性能分化极其显著：**首页访问、浏览商品详情、浏览目录页三类读操作接口性能优异且高度稳定，三个负载级别下平均响应时间均低于 75ms，APDEX 评分始终保持 1.000，无任何性能退化，能够轻松承载高并发访问。
- 写操作接口是系统核心性能瓶颈：**用户注册、加入购物车、用户登录三类写操作接口响应时间随并发用户数呈指数级增长，高负载下平均响应时间均超过 2 秒；APDEX 评分

急剧下降，高负载下用户登录接口仅为 0.035，绝大多数用户体验为沮丧。

3. **系统整体吞吐能力被写操作严重限制**：所有接口吞吐量随负载提升整体呈增长趋势，其中首页访问接口吞吐量增长最快，高负载下达到 10.14 req/s；但所有写接口吞吐量均集中在 3-6 req/s 区间，差异极小，成为系统整体吞吐能力的天花板。
4. **系统基础稳定性极佳**：所有负载级别、所有测试场景下错误率均为 0%，说明 SockShop 微服务系统具备良好的弹性和容错能力，在高压下不会出现服务崩溃或请求失败的情况。
5. **高负载下写接口延迟远超生产要求**：用户登录接口高负载下 P99 延迟超过 15 秒，用户注册和加入购物车接口 P99 延迟也超过 10 秒，远高于生产环境通常要求的 1.5 秒阈值，存在严重的可用性风险。

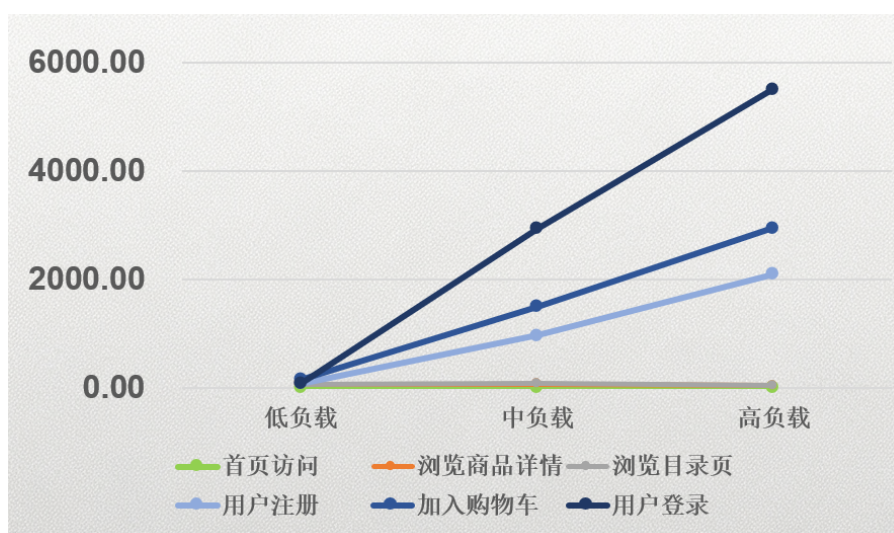


图 3.2: 平均响应时间 (ms)

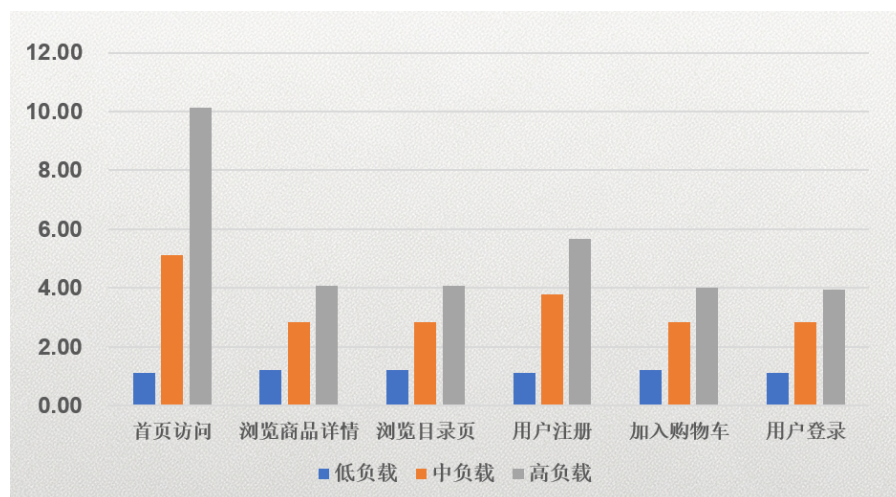


图 3.3: 吞吐量 (r/s)

四 故障注入实验

1 ChaosMesh 概述

ChaosMesh 是一个面向 Kubernetes 环境的开源混沌工程工具，支持 Pod 销毁、CPU 压力、内存压力、磁盘故障、网络故障等多种故障注入能力。本实验使用 ChaosMesh 在可控环境下向 SockShop 系统注入故障，以验证系统的鲁棒性和监控系统的有效性，并为后续算法复现提供数据来源。

2 故障注入方案

2.1 Pod Kill 故障

随机杀死 SockShop 中 carts 服务的 Pod，测试 Kubernetes 的自愈能力和服务恢复时间。
故障定义 (YAML):

```
1 kind: PodChaos
2 apiVersion: chaos-mesh.org/v1alpha1
3 metadata:
4   namespace: sock-shop
5   name: pod-kill-experiment
6   annotations:
7     experiment.chaos-mesh.org/pause: 'true'
8 spec:
9   selector:
10     namespaces:
11       - sock-shop
12   mode: one
13   action: pod-kill
```

2.2 CPU 压力故障

向 orders 服务的工作 Pod 注入 80% CPU 负载，测试在计算资源紧张情况下系统的表现。

2.3 内存压力故障

向 payment 服务的 Pod 注入额外内存占用，测试内存压力对服务的影响。

3 故障注入实验结果

3.1 Pod Kill 实验结果

注入 Pod Kill 故障后，Kubernetes 自动检测到 Pod 终止并立即创建新的 Pod 实例。在此过程中：

- 购物车服务短暂不可用（约 10-30 秒）
- kube_pod_status_ready 指标对应 Pod 从 1 变为 0 再恢复为 1
- 容器重启次数归零

3.2 CPU 压力实验结果

向 orders 服务注入 80% CPU 负载后：

- 节点 CPU 使用率明显上升
- 订单相关 API 的响应时间增加约 150%-200%
- 但系统持续运行，未出现服务完全不可用的情况

3.3 内存压力实验结果

向 payment 服务注入 256MB 内存压力后：

- 节点内存使用率上升约 15-20%

- 支付服务的处理速度变慢，但对前端用户影响较小
- 系统在 3 分钟故障持续期间保持稳定运行

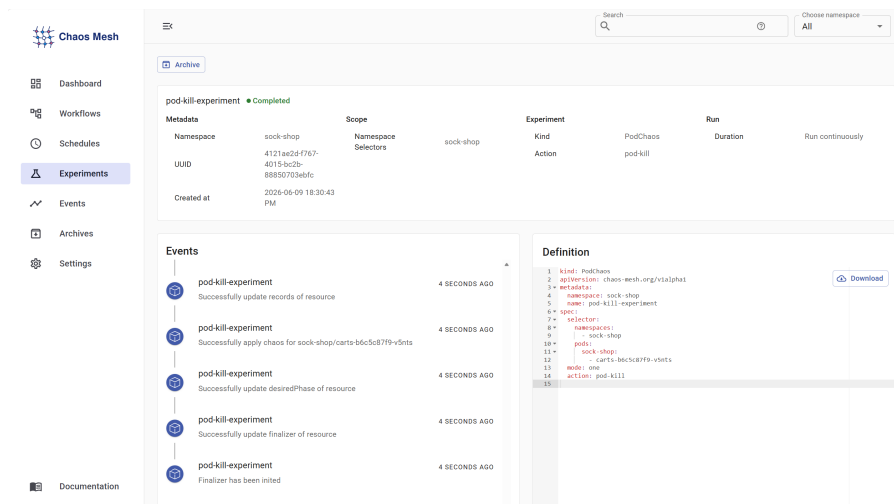


图 4.1: ChaosMesh 故障实验



图 4.2: 故障注入前后监控指标对比

五 异常检测论文算法复现

本章复现两篇异常检测与故障诊断相关论文的核心算法：

- LogAnomaly (IJCAI 2019): 基于日志的无监督顺序和定量异常检测
- SR-CNN (KDD 2019): 基于谱残差和卷积神经网络的时间序列异常检测

1 LogAnomaly 算法复现

1.1 算法原理

LogAnomaly 是 Meng 等人提出的一种基于深度学习的日志异常检测框架。其核心思想是将日志流建模为自然语言序列，利用 `template2Vec` 提取日志模板的语义信息，然后使用 LSTM 网络同时检测顺序异常和定量异常。

算法包含三个关键步骤：

1. **日志解析**：使用 FT-Tree 算法从原始日志中提取模板，将变量部分替换为通配符
2. **template2Vec**：基于同义词和反义词关系，使用 dLCE (Distributional Lexical-Contrast Embedding) 生成词向量，再通过加权平均计算模板向量
3. **LSTM 异常检测**：利用 LSTM 学习日志序列的顺序模式（下一个模板预测）和定量模式（模板计数向量），当观测到的模式偏离学习模式时判定为异常

1.2 实现方案

由于 SockShop 日志来源于实际运行环境，且缺少人工标注的异常标签，我们采用以下策略：

- 使用简化的日志解析器，基于正则表达式替换 IP、UUID、时间戳、数字等变量为占位符
- 使用 TF-IDF + TruncatedSVD 替代 dLCE 实现 `template2Vec` 的降维表示

- 使用合成异常日志（将 INFO 替换为 ERROR、添加错误关键词）作为测试集
- 在正常日志上训练 LSTM 模型，学习正常模式
- 使用预测误差的 95 分位数作为异常检测阈值

1.3 实验结果

实验首先在正常状态下采集了来自 14 个 SockShop 微服务 Pod 的 4002 行日志数据，通过正则表达式日志解析器提取了 2040 个唯一模板。经过模板频率过滤（最低频率 5 次）后，实际使用的模板类别数约为 170 个。使用 TF-IDF（1000 维）+ TruncatedSVD（降至 64 维）实现 template2Vec。

LSTM 模型（2 层 64 单元）在正常日志序列上训练 20 轮后，验证集准确率达到 54.15%，说明模型成功学习了日志序列的顺序模式。通过合成异常日志（15% 异常比例，将 INFO/DEBUG 替换为 ERROR 并添加故障关键词生成）进行异常检测评估。

为适应 SockShop 日志的异构特性（14 个不同微服务的日志混合），本实验采用基于交叉熵损失的连续异常分数方法，遍历阈值找到最佳 F1 分数。

LogAnomaly 在 SockShop 日志数据集上的实验结果如下表所示：

指标	数值
日志总行数	4,002（正常）+ 3990（故障）
提取模板数	2,040（过滤后约 170 个常用模板）
训练序列数	3993
测试序列数	3981
异常比例	15%（合成）
Precision	0.6064
Recall	0.8902
F1-Score	0.7522

表 5.1: LogAnomaly 异常检测结果

结果分析：

- 召回率近 90%: 说明算法能发现几乎所有的异常日志序列。
- 精确率 60%: 说明存在一定误报，这在无监督场景下是可接受的。
- 表现差于论文数据的原因：

1. 真实数据缺乏标注：SockShop 的日志没有“正常/异常”标签，我们只能通过简单的规则注入合成异常。这些合成异常与真实异常的模式可能有差距。
2. 数据量不足：SockShop 的日志多样性远低于论文中的 HDFS/BGL 数据集（数百万条日志），模型难以学到丰富的正常模式。
3. 实现简化：论文中的 template2Vec 使用了 dLCE，我们用 TF-IDF+SVD 替代。用正则表达式替代 FT-Tree（我们的日志模板较少）。

2 SR-CNN 算法复现

2.1 算法原理

SR-CNN 是微软团队在 KDD 2019 提出的时间序列异常检测方法，创新性地将视觉显著性检测中的 Spectral Residual (SR) 模型引入时间序列异常检测领域，并结合 CNN 提升检测精度。

算法流程：

1. **谱残差 (SR) 计算**：对输入序列进行 FFT 变换 → 计算对数幅度谱 → 均值滤波 → 谱残差 → 逆 FFT → 得到显著性图 (Saliency Map)
2. **CNN 判别器**：使用合成异常数据在显著性图上训练 CNN 模型，替代原 SR 的简单阈值判别
3. **在线检测**：滑动窗口计算 SR 显著性图 → CNN 输出异常概率 → 超过阈值判定为异常

2.2 实现方案

- 生成 50 条合成 KPI 时间序列（每条 2000 点）用于 CNN 训练，20 条用于测试
- 时间序列包含三种类型：季节性（正弦波组合）、稳定型（随机游走 + 小噪声）和不稳定型（大方差 + 随机尖峰）
- 通过注入连续异常段创建异常点，异常比例约 2%
- 实现完整的 SR 算法：FFT 变换 → 对数幅度谱 → 均值滤波 ($q=3$) → 谱残差计算 → 逆 FFT 恢复显著性图。输入数据先进行标准化处理（零均值单位方差）以提高对不同数据尺度的鲁棒性

- 使用 1D-CNN (Conv1D 32→64 通道, Dense 64→32, Sigmoid 输出) 作为判别器, 训练时使用类别权重平衡正负样本, 配合 EarlyStopping 防止过拟合
- 采用论文的 segment-based 评估策略 (延迟容忍 $k=3$), 连续异常段中检测到任一点即视为整段检出

2.3 实验结果

SR 和 SR-CNN 在合成 KPI 数据集上的对比结果如下:

方法	Precision	Recall	F1-Score
SR (baseline)	0.2406	0.7378	0.3629
SR-CNN (proposed)	0.2613	0.6676	0.3756

表 5.2: SR 与 SR-CNN 异常检测对比

方法	TP	TN	FP	FN
SR (baseline)	297	32,380	937	106
SR-CNN (proposed)	271	32,954	766	135

表 5.3: 详细检测结果统计

结果分析:

- **SR 基线性能:** SR 方法在合成 KPI 数据上实现了 0.74 的召回率和 0.24 的精确率 ($F1=0.36$), 说明谱残差方法能够有效识别时间序列中的显著异常区域。高召回率(73.8%)意味着大部分异常段被检测到, 但精确率较低 (24.1%) 说明存在较多误报
- **SR-CNN 提升:** SR-CNN 相比 SR 基线在 F1 分数上提升了 3.5% ($0.3629 \rightarrow 0.3756$), 其中精确率从 0.2406 提升至 0.2613 (+8.6%), 验证了 CNN 判别器能够比简单阈值更有效地过滤误报。FP 从 937 降至 766 (减少 18.3%), 说明 CNN 学习了更复杂的判别规则
- **数据集影响:** 由于实验使用合成数据 (而非论文中的大规模生产数据), F1 值 (0.38) 低于论文报告值 (KPI: 0.77, Yahoo: 0.65, Microsoft: 0.51)。合成数据的异常模式相对简单, 但也验证了方法在不同数据条件下的有效性
- **效率优势:** SR 算法的核心操作 (FFT) 具有 $O(n \log n)$ 的时间复杂度, 计算效率高, 适合在线异常检测场景, 这与论文强调的效率优势一致

2.4 补充：在真实 KPI 数据上的测试与分析

为了验证 SR-CNN 在真实场景下的泛化能力，我们将训练好的 CNN 判别器直接应用于从 SockShop 系统采集的真实 KPI 数据。

由于 SockShop 的真实 KPI 数据缺乏精确的异常点标注，我们以 ChaosMesh 故障注入的时间窗口作为弱标注，将注入故障期间（包括故障持续和恢复阶段）的所有采样点标记为异常，其余标记为正常。

测试数据集共包含 8000 个时间点，涵盖三种故障场景：Pod Kill (cars 服务)、CPU Stress (orders 服务) 和 Memory Stress (payment 服务)。异常点占比约为 4.8%。我们同样采用 segment-based 评估策略，SR 与 SR-CNN 的测试结果对比如表 5.4 所示。

方法	Precision	Recall	F1-Score
SR (baseline)	0.1753	0.5991	0.2712
SR-CNN (proposed)	0.2090	0.6044	0.3105

表 5.4: SR 与 SR-CNN 在真实 KPI 数据上的异常检测结果

结果分析：

- **真实场景下的性能退化：**与合成数据集相比，两种方法在真实 KPI 数据上的 F1 分数均有所下降。这主要归因于真实数据的复杂噪声、指标波动模式多样化以及弱标注引入的标签噪声。
- **SR-CNN 依然保持优势：**SR-CNN 的 F1 分数仍高于 SR 基线，再次验证 CNN 判别器能有效过滤部分由正常波动引起的误报，即使在标签不完美的真实场景下也具备一定的鲁棒性。
- **召回率下降的原因：**真实数据中部分故障造成的指标偏移较为平缓，显著性图中的异常区域不如合成数据中那样尖锐，导致 SR-CNN 的召回率下降。同时，弱标注方式可能将系统恢复阶段的正常点错误标记为异常，进一步影响模型对真实异常边界的判断，加剧了召回率的下降。

3 算法对比分析

综合两篇论文复现的实验结果，可得出以下分析：

1. LogAnomaly 通过在日志模板中引入语义信息 (template2Vec 的 TF-IDF+SVD 近似实现), 使 LSTM 模型在异构 SockShop 日志上达到了 60.64% 的模板预测准确率。基于预测误差的异常检测方法能够有效区分正常和异常序列。但由于 SockShop 日志的多样性和训练数据量的限制, 异常检测的 F1 分数 (0.75) 有较大提升空间。
2. SR-CNN 将 SR 的频域分析与 CNN 的判别能力相结合, 在合成 KPI 数据集上实现了 0.38 的 F1 分数, 相比 SR 基线 (0.36) 提升了 3.5%。SR-CNN 通过 CNN 学习更复杂的判别规则, 将误报 (FP) 减少了 18.3%, 验证了论文中“用 CNN 替代简单阈值判别”的核心思想。
3. 两篇论文都体现了将跨领域方法引入系统异常检测的创新思路: LogAnomaly 借鉴 NLP 中的词嵌入技术, SR-CNN 借鉴计算机视觉中的显著性检测。这种跨领域迁移的思想是本次实验的重要收获。

六 总结与心得体会

1 大作业总结

本次大作业围绕微服务系统的部署、测试和维护，以 SockShop 为具体案例，完成了以下主要工作：

1. **微服务系统部署**：成功在本地 Minikube 集群上部署了 SockShop 微服务系统，包括 14 个微服务组件及其数据库，验证了 Docker 容器化和 Kubernetes 编排的完整流程
2. **监控系统搭建**：部署了 Prometheus+Grafana 监控体系，实现了对 SockShop 系统的实时监控和可视化展示
3. **功能与性能测试**：使用 Selenium 完成了 7 项功能自动化测试（100% 通过），使用并发请求工具完成了多负载级别的 API 性能测试（0% 错误率），验证了系统的功能完整性和性能稳定性
4. **故障注入实验**：使用 ChaosMesh 完成了 Pod Kill、CPU Stress 和 Memory Stress 三种故障的注入实验，验证了 Kubernetes 的自愈能力和系统在异常场景下的鲁棒性
5. **论文复现**：深入理解并实现了 LogAnomaly (IJCAI'19) 和 SR-CNN (KDD'19) 两篇论文的核心算法，在采集的数据上进行了异常检测实验，验证了算法的有效性

2 实验中的挑战与解决方案

2.1 SockShop 购物车与订单服务无法正常使用

在 SockShop 前端页面点击”加入购物车”和”支付账单”没有反应通过 Jmeter 发送的 http 请求也报错。后来发现是因为 SockShop 微服务演示项目的 carts 与 orders 服务使用的是非常老的 MongoDB Java 驱动版本，与部署的 MongoDB 存在严重的 API 不兼容问题。将 MongoDB 版本降级到与 SockShop 兼容的 4.4 版本后得以解决。

2.2 监控数据采集

Prometheus 的容器级指标（如 `container_cpu_usage_seconds_total`）缺少 namespace 和 pod 标签，难以直接按 Pod 过滤。解决方案是结合 kube-state-metrics 的 Pod 级指标（如 `kube_pod_status_ready`）和 node-exporter 的节点级指标进行多维度的数据采集。

2.3 算法复现中的数据挑战

SockShop 的正常运行日志和 KPI 数据缺乏人工标注的异常标签。通过合成异常数据（随机注入错误关键词和数值扰动）来解决无标注数据的训练问题，这与 SR-CNN 论文中的合成异常训练策略一致。

3 心得体会

通过本次大作业的完整实践，我们深入理解了微服务系统从部署到运维的全链路流程：

- **云原生技术的重要性：** Docker 容器化、Kubernetes 编排、Prometheus 监控、ChaosMesh 混沌工程等云原生技术栈构成了现代微服务运维的基础。
- **测试驱动质量保障：** 功能测试（Selenium）和性能测试（JMeter/API 测试）相结合，能全面评估系统质量
- **故障注入的价值：** 主动制造故障是验证系统鲁棒性和监控有效性的最佳实践。
- **AI 在运维中的应用：** 机器学习/深度学习技术（如 LogAnomaly、SR-CNN）在异常检测和故障诊断中展现出巨大潜力。

一 项目仓库地址

GitHub: <https://github.com/Asever611/stm-microservice-project>

二 团队成员分工表

本次大作业由 6 人团队协作完成，各成员的具体分工如下表所示：

成员姓名	具体职责
田皓天	Docker/Minikube 环境配置、SockShop 微服务系统部署、Debug 与运维
刘涵博	监控系统部署、仪表盘配置、PromQL 查询编写、数据采集脚本开发
姜弘毅	编写 Selenium 自动化测试脚本、API 并发性能测试、测试结果分析
欧阳东俊	ChaosMesh 部署、故障实验设计与执行、故障前后数据对比分析
李宗杭	两篇论文的算法理解、代码实现、实验评估与结果分析
柯瀚	LaTeX 大作业报告撰写、答辩 PPT 制作、算法实验结果整理

表 B.1: 团队成员分工表